

Software Development Life Cycle (SDLC)

Presented by: Dr. Abdolraheem Khader
May 2025

Introduction to SDLC

- Definition: A systematic process for planning, creating, testing, and deploying an information system.
- Importance: Ensures structured and efficient software development.
- Key Objective: To deliver high-quality software that meets or exceeds customer expectations.

Why Use SDLC?

- Improves software quality and efficiency.
- Reduces project risks and errors.
- Enhances project management and progress tracking.
- Facilitates collaboration among stakeholders.

Phases of SDLC (Overview)

- Requirements Gathering.
- System Design.
- Implementation (Coding).
- Testing.
- Deployment.
- Maintenance.
- (Include a flowchart diagram).

Phase 1 - Requirements Gathering

- Objective: Understand the client's needs and expectations.
- Techniques: Interviews, questionnaires, observation, and document analyses.
- Deliverable: Requirement Specification Document (RSD).
- Example: Gathering user requirements for a mobile banking app.

Phase 1 - Requirements Gathering (Challenges)

- Ambiguity in client expectations.
- Incomplete or conflicting requirements.
- Techniques to Overcome: Stakeholder meetings, requirement validation, and prioritization.

Phase 2 - System Design

- Purpose: Create a blueprint for the system architecture.
- Components: High-Level Design (HLD) and Low-Level Design (LLD).
- Deliverables: System Architecture Diagrams, Data Flow Diagrams (DFD).
- Example: Designing a multi-tier architecture for an e-commerce platform.

Phase 2 - Design Models

- Waterfall: Sequential, phase-wise design.
- Agile: Incremental and iterative.
- Spiral: Risk-driven with iterative prototypes.
- V-Model: Verification and validation at each stage.

Phase 3 - Implementation (Coding)

- Translating design into executable code.
- Programming Languages: Python, Java, C++, etc.
- Coding Standards: Readability, maintainability, and efficiency.
- Example: Developing a REST API for an online store.

Phase 3 - Version Control in Implementation

- Git and GitHub for collaborative coding.
- Importance of version control: Track changes, rollback, and collaboration.
- Best Practices: Frequent commits, descriptive messages, and branching.

Phase 4 - Testing

- Purpose: Identify and fix software defects.
- Types:
 - ✓ Unit Testing
 - ✓ Integration Testing
 - ✓ System Testing
 - ✓ Acceptance Testing
- Automated vs. Manual Testing (**HW**).

Phase 4 - Testing Strategies

- Test-Driven Development (TDD): Writing tests before code.
- Continuous Integration Testing: Automated tests with every code push.
- Example: Unit tests for a login module.

Phase 5 - Deployment

-
- Releasing the software to the production environment.
 - Techniques:
 - Continuous Deployment.
 - Staged Rollout.
 - Deployment Platforms: Cloud, On-Premises.
 - Example: Deploying a web application using Docker.

Phase 5 - Deployment Challenges

- Environment Compatibility Issues.
- Downtime and Rollback Mechanisms.
- User Training and Documentation.
- Solutions: CI/CD pipelines and containerization.

Phase 6 - Maintenance

- Types:
 - Corrective: Fixing bugs.
 - Adaptive: Modifying for new environments.
 - Perfective: Improving performance.
 - Preventive: Anticipating future issues.
- Example: Upgrading database compatibility.

SDLC Models: An Overview

- Waterfall Model.
- Agile Model.
- Spiral Model.
- V-Model.
- Big Bang Model.
- Incremental Model.

SDLC Models: Waterfall Model

- Characteristics: Sequential and linear.
- Best for: Well-defined requirements.
- Disadvantage: Inflexibility to changes.
- Example: Government or military software systems.

SDLC Models: Agile Model

- Characteristics: Iterative and flexible.
- Best for: Rapid development and changing requirements.
- Disadvantage: Can be chaotic without proper management.
- Example: Software startups and mobile app development.

SDLC Models: Spiral Model

- Characteristics: Risk-driven and iterative.
- Best for: Complex, high-risk projects.
- Disadvantage: Expensive and time-consuming.
- Example: Large-scale enterprise applications.

SDLC Models: V-Model

- Characteristics: Validation and verification at each stage.
- Best for: Critical systems (e.g., healthcare).
- Disadvantage: Rigid and costly.
- Example: Medical software systems.

SDLC Models: Incremental Model

- Characteristics: Partial implementation followed by enhancements.
- Best for: Gradually evolving systems.
- Example: Enterprise resource planning (ERP) systems.

Case Study: SDLC in E-Commerce Development

- Problem: Building a scalable online shopping platform.
- SDLC Approach: Agile.
- Outcome: Incremental feature releases and fast updates.

Common Challenges in SDLC

- Changing Requirements.
- Communication Gaps.
- Budget and Time Constraints.
- Quality Assurance Issues.
- Solutions: Agile practices, stakeholder involvement, automated testing.

Best Practices for Effective SDLC

- Clear Requirement Documentation.
- Early Risk Identification.
- Frequent Testing and Iteration.
- Stakeholder Collaboration.

Tools for SDLC

- Requirements: JIRA, Confluence.
- Design: Lucidchart, Microsoft Visio.
- Coding: Git, Visual Studio Code.
- Testing: Selenium, JUnit.
- Deployment: Jenkins, Docker.
- Maintenance: Splunk, Nagios.

Real-World Example: SDLC in Banking Software

- Goal: Secure and reliable transaction processing.
- Model: V-Model due to critical data handling.
- Outcome: Robust and secure application.

Workshop Activity

- Task: Identify the suitable SDLC model for a mobile healthcare app.
- Discuss: Pros and cons of the chosen model..

Thank You!